



Universidad Nacional de Río Cuarto

Facultad de Ciencias Exactas, Físico-Químicas y Naturales

Simulación – Prof. Ariel González

Proyecto de Simulación

Sistema de Tiempo Real: Controlador Simplificado de Bomba
de Infusión

Simulación de Sistemas

Autor:

Emiliano Bernal

Río Cuarto, Córdoba
Junio de 2026

Índice

1. Descripción general	3
2. Objetivo del proyecto	3
3. Entradas del sistema	3
4. Salidas del sistema	4
5. Requisitos temporales	4
6. Parámetros del sistema	5
7. Componentes propuestos	5
7.1. Generador de órdenes médicas	5
7.2. Sensor de flujo	5
7.3. Controlador de bomba	5
7.4. Actuador de la bomba	6
7.5. Módulo de alarmas	6
8. Modelos atómicos DEVS	6
8.1. Generador de órdenes médicas	6
8.2. Sensor de flujo	7
8.3. Controlador de bomba	8
8.4. Actuador de la bomba	13
8.5. Módulo de alarmas	14
8.6. Registro de eventos	16
8.7. Bolsa	19
8.8. Enfermero	20
9. Modelo acoplado	22
9.1. Especificación formal del acoplamiento	22
10.Implementación en PowerDEVS	25
10.1. Estructura general del modelo	25
10.2. Organización de los archivos	29
10.3. Parametrización del sistema	30
11.Resultados de Simulación y Análisis	31
11.1. Funcionamiento Normal sin fallas	32
11.2. Cambio de orden médica durante la infusión	32
11.3. Orden médica con caudal igual a cero	33
11.4. Desvío leve de caudal	35
11.5. Desvío de caudal mayor al 10 %	36
11.6. Fin de bolsa con confirmación del enfermero	38
11.7. Alarma crítica no confirmada durante 30 segundos	40
12.Propiedades verificadas	42

13.Métricas obtenidas	42
14.Propuesta de mejora	44
15.Conclusiones	45

1. Descripción general

Se desea modelar y simular un sistema automático de administración de medicación intravenosa. El sistema controla una bomba de infusión que debe suministrar una droga a un paciente respetando una dosis objetivo indicada por una orden médica.

En esta versión reducida del proyecto se considerará un controlador simplificado. El sistema debe recibir órdenes médicas, controlar el caudal administrado, detectar desviaciones persistentes respecto del caudal indicado y reaccionar ante la condición de fin de bolsa. También deberá gestionar alarmas asociadas a dichas situaciones.

El objetivo principal es estudiar el comportamiento temporal del sistema y verificar que las acciones de control ocurran dentro de los tiempos especificados.

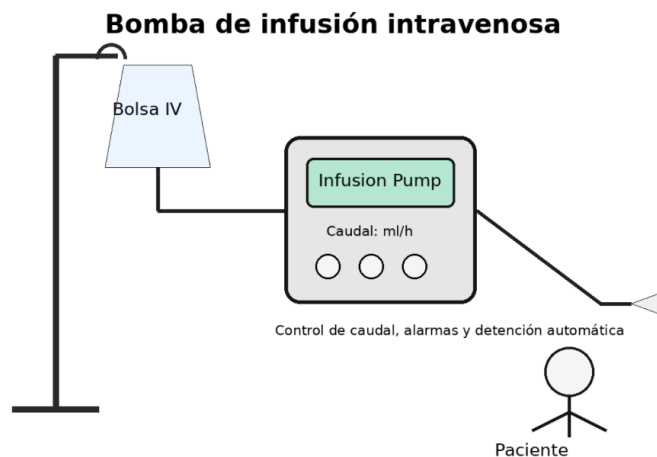


Figura 1: Esquema de una bomba de infusión intravenosa.

2. Objetivo del proyecto

El objetivo del proyecto es construir un modelo de simulación de eventos discretos que represente el comportamiento de una bomba de infusión simplificada. Se deben considerar escenarios normales de funcionamiento y escenarios de falla asociados al caudal administrado y al agotamiento de la bolsa de medicación.

3. Entradas del sistema

El sistema recibe los siguientes eventos de entrada:

- **ordenMedica:** indica el caudal objetivo que debe administrarse, expresado en ml/h. Las órdenes médicas podrán indicar caudales comprendidos entre 0 y 200 ml/h. Una orden con caudal igual a cero indica la detención de la infusión.
- **sensorFlujo:** informa periódicamente el caudal real administrado.
- **finBolsa:** indica que la bolsa de medicación está próxima a agotarse.
- **confirmacionEnfermero:** indica que una alarma fue reconocida por personal médico.

4. Salidas del sistema

Las salidas posibles del sistema son:

- **ajustarCaudal**: ordena modificar el caudal de infusión.
- **detenerBomba**: detiene la infusión.
- **alarmaBaja**: notificación preventiva o de baja criticidad.
- **alarmaMedia**: notificación por desviaciones sostenidas del funcionamiento esperado.
- **alarmaCritica**: notificación ante una condición que requiere atención inmediata.
- **registrarEvento**: registra el evento ocurrido en el registro de eventos del sistema.

5. Requisitos temporales

El sistema debe cumplir los siguientes requisitos temporales:

1. Luego de recibir una **ordenMedica** con caudal mayor que cero, la bomba debe comenzar la infusión en menos de 3 segundos.
2. Una orden médica especifica únicamente el caudal de infusión. La bomba mantiene dicho caudal hasta recibir una nueva **ordenMedica**. Una orden con caudal igual a cero indica la detención de la infusión.
3. El sensor de flujo debe informar el caudal real cada 1 segundo.
4. Si el caudal real difiere en más de un 10 % respecto del caudal indicado durante más de 5 segundos, el sistema debe emitir una **alarmaMedia**.
5. Si el sistema no logra corregir el desvío de caudal luego de emitir una **alarmaMedia**, podrá detener la bomba y emitir una **alarmaCritica**. El criterio exacto para pasar de **alarmaMedia** a **alarmaCritica** deberá ser definido por cada grupo.
6. Si una **alarmaCritica** no es confirmada por el enfermero dentro de 30 segundos, la alarma debe repetirse cada 10 segundos.
7. Si se detecta **finBolsa**, debe emitirse una **alarmaBaja** y la infusión sólo puede continuar durante un máximo de 60 segundos. Transcurridos los 60 segundos, la bomba debe detener automáticamente la infusión.
8. Luego de una **alarmaCritica**, la bomba permanecerá detenida hasta recibir una **confirmacionEnfermero** o una nueva **ordenMedica**.

6. Parámetros del sistema

Parámetro	Valor sugerido
Rango de caudal permitido	0 a 200 ml/h
Tiempo máximo para iniciar infusión	3 s
Período de muestreo del sensor de flujo	1 s
Latencia del actuador de la bomba	0.5 s
Desvío máximo permitido de caudal	10 %
Tiempo máximo de desvío de caudal	5 s
Tiempo para confirmar alarma crítica	30 s
Período de repetición de alarma crítica	10 s
Tiempo máximo luego de fin de bolsa	60 s

Cuadro 1: Parámetros temporales del sistema.

7. Componentes propuestos

El sistema puede modelarse mediante los siguientes modelos atómicos DEVS.

7.1. Generador de órdenes médicas

Genera eventos **ordenMedica** que indican el caudal objetivo que debe administrar la bomba. Las órdenes pueden generarse de manera determinística o estocástica, según los escenarios definidos por el grupo.

Salida: ordenMedica.

7.2. Sensor de flujo

Mide periódicamente el caudal real administrado por la bomba.

Entrada: caudalActual.

Salida: sensorFlujo.

Este sensor debe emitir una medición cada 1 segundo.

7.3. Controlador de bomba

Es el componente central del sistema. Recibe las órdenes médicas, las mediciones del sensor de flujo, la señal de fin de bolsa y las confirmaciones del enfermero. Decide si debe iniciar la infusión, ajustar el caudal, detener la bomba o emitir alarmas.

Entradas:

- ordenMedica
- sensorFlujo
- finBolsa
- confirmacionEnfermero

Salidas:

- ajustarCaudal
- detenerBomba
- alarmaBaja
- alarmaMedia
- alarmaCritica
- registrarEvento

7.4. Actuador de la bomba

Representa el mecanismo físico que administra la medicación al paciente. Recibe órdenes del controlador y modifica el caudal real.

Entradas: ajustarCaudal, detenerBomba.

Salida: caudalActual.

La latencia máxima para aplicar un ajuste de caudal es de 0.5 segundos.

7.5. Módulo de alarmas

Gestiona las alarmas visuales y sonoras. Debe repetir las alarmas críticas no confirmadas.

Entradas: alarmaBaja, alarmaMedia, alarmaCritica, confirmacionEnfermero.

Salida: notificacionAlarma.

8. Modelos atómicos DEVS

En esta sección se presentan los modelos atómicos del sistema utilizando una notación similar a CML-DEVS. Cada modelo se especifica mediante sus conjuntos de entrada, salida, estado, funciones de transición, función de salida y función de avance de tiempo.

8.1. Generador de órdenes médicas

atomic GeneradorOrdenes is $\langle Y, S, \delta_{int}, \lambda, ta \rangle$ where

S is

caudalObjetivo : [0,200];

σ : Time;

end S

Y is

out : {ordenMedica} x [0,200];

end Y

δ_{int} (caudalObjetivo, σ) is

caudalObjetivo = siguiente_caudal();

σ = siguiente_tiempo();

end δ_{int}

```

λ(caudalObjetivo, σ) is
    out = (ordenMedica, caudalObjetivo);
end λ

```

```

ta(caudalObjetivo, σ) is
    σ;
end ta

```

```

end atomic

```

Estado inicial: (100, 0).

$$S = (\text{caudalObjetivo}, \sigma)$$

donde *caudalObjetivo* es el valor de caudal que será enviado en la próxima orden médica y σ es el tiempo restante hasta la próxima generación de una orden médica.

8.2. Sensor de flujo

atomic SensorFlujo is <X, Y, S, δ_{int} , δ_{ext} , λ , ta> where

```

S is
    caudalReal : ℝ;
    σ : Time;
end S

```

```

X is
    in : {caudalActual} x ℝ;
end X

```

```

Y is
    out : {sensorFlujo} x ℝ;
end Y

```

```

δext((caudalReal, σ), e, (in,x)) is
    caudalReal = x.getSnd();
    σ = max(0, σ - e);
end δext

```

```

δint(caudalReal, σ) is
    σ = 1;
end δint

```

```

λ(caudalReal, σ) is
    out = (sensorFlujo, caudalReal);
end λ

```

```

ta(caudalReal, σ) is
    σ;
end ta

```


end atomic

Estado inicial: $(0,1)$.

$$S = (caudalReal, \sigma)$$

donde *caudalReal* es el caudal real administrado por la bomba y σ es la variable de avance del tiempo.

Además, se considera que `getFt()` devuelve el primer componente de una tupla y `getSnd()` devuelve el segundo componente de una tupla.

8.3. Controlador de bomba

atomic ControladorBomba is $\langle X, Y, S, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$ where

S is

```
fase : {0,1,2};
cObj  : [0,200];
cReal : [0,200];
eb    : {0,1};
ea    : {0,1,2,3};
td    : Time;
tb    : Time;
σ     : Time;
```

end S

X is

```
in : ({ordenMedica} x [0,200])
    ∪ ({sensorFlujo} x [0,200])
    ∪ {finBolsa, confirmacionEnfermero};
```

end X

Y is

```
out : ({ajustarCaudal} x [0,200])
      ∪ {detenerBomba, alarmaBaja, alarmaMedia, alarmaCritica};
```

end Y

$\delta_{ext}((fase, cObj, cReal, eb, ea, td, tb, \sigma), e, (in, x))$ is

defcases

case

```
fase = 1;
cObj = x.getSnd();
ea = 0;
td = 0;
σ = 0;
```

if

```
x.getFt() = ordenMedica and x.getSnd() > 0;
```

case

```
fase = 0;
cObj = 0;
ea = 0;
```

```

    td = 0;
     $\sigma$  = 0;
if
    x.getFt() = ordenMedica and x.getSnd() = 0;

case
    fase = 2;
    cReal = x.getSnd();
    td = td + e;
     $\sigma$  = max(0, 5 - td);
if
    x.getFt() = sensorFlujo and abs(x.getSnd() - cObj) > 0.1 * cObj and fase
    = 2 and ea = 0;

case
    fase = 2;
    cReal = x.getSnd();
    td = 0;
     $\sigma$  = 5;
if
    x.getFt() = sensorFlujo and abs(x.getSnd() - cObj) > 0.1 * cObj and fase
    != 2;

case
    fase = 1;
    cReal = x.getSnd();
    ea = 0;
    td = 0;
     $\sigma$  =  $\infty$ ;
if
    x.getFt() = sensorFlujo and abs(x.getSnd() - cObj) <= 0.1 * cObj;

case
    fase = 2;
    cReal = x.getSnd();
    td = td + e;
     $\sigma$  = max(0, 5 - td);
if
    x.getFt() = sensorFlujo and abs(x.getSnd() - cObj) > 0.1 * cObj and ea =
    2;

case
    eb = 1;
    ea = 1;
    tb = 0;
     $\sigma$  = 0;
if
    x = finBolsa;

case
    fase = 0;

```

```

    ea = 0;
     $\sigma$  =  $\infty$ ;
if
    x = confirmacionEnfermero and ea = 3 and fase = 0;

case
    ea = 0;
if
    x = confirmacionEnfermero;
end defcases
end  $\delta_{ext}$ 

 $\delta_{int}((\text{fase}, cObj, cReal, eb, ea, td, tb, \sigma))$  is
defcases
case
    fase = 1;
    ea = 0;
    td = 0;
     $\sigma$  =  $\infty$ ;
if
    fase = 1 and cObj > 0;

case
    fase = 0;
    cObj = 0;
    ea = 0;
    td = 0;
     $\sigma$  =  $\infty$ ;
if
    fase = 0 and cObj = 0;

case
    fase = 2;
    ea = 2;
    td = 0;
     $\sigma$  = 5;
if
    fase = 2 and ea = 0;

case
    fase = 0;
    ea = 3;
    td = 0;
     $\sigma$  =  $\infty$ ;
if
    fase = 2 and ea = 2;

case
    fase = fase;
    eb = 1;
    ea = 1;

```

```

        td = td;
        tb = 60;
         $\sigma$  = 60;
    if
        eb = 1 and ea = 1 and tb = 0;

    case
        fase = 0;
        cObj = 0;
        eb = 1;
        ea = 1;
        tb = 60;
         $\sigma$  =  $\infty$ ;
    if
        eb = 1 and ea = 1 and tb = 60;
    end defcases
end  $\delta_{int}$ 

 $\lambda((\text{fase}, \text{cObj}, \text{cReal}, \text{eb}, \text{ea}, \text{td}, \text{tb}, \sigma))$  is
defcases
    case
        out = alarmaBaja;
    if
        eb = 1 and ea = 1 and tb = 0;

    case
        out = detenerBomba;
    if
        eb = 1 and ea = 1 and tb = 60;

    case
        out = (ajustarCaudal, cObj);
    if
        fase = 1 and cObj > 0 and ea = 0;

    case
        out = detenerBomba;
    if
        fase = 0 and cObj = 0;

    case
        out = alarmaMedia;
    if
        fase = 2 and ea = 0;

    case
        out = alarmaCritica;
    if
        fase = 2 and ea = 2;
    end defcases
end  $\lambda$ 

```

```

ta((fase,cObj,cReal,eb,ea,td,tb,σ)) is
    σ;
end ta

```

```

end atomic

```

Estado inicial: $(0, 0, 0, 0, 0, 0, 0, \infty)$.

$$S = (fase, cObj, cReal, eb, ea, td, tb, \sigma)$$

donde:

- $fase = 0$: detenida.
- $fase = 1$: infundiendo.
- $fase = 2$: corrigiendo.
- $cObj$: caudal objetivo indicado por la última orden médica.
- $cReal$: caudal real informado por el sensor de flujo.
- $eb = 0$: bolsa normal.
- $eb = 1$: fin de bolsa.
- $ea = 0$: sin alarma.
- $ea = 1$: alarma baja.
- $ea = 2$: alarma media.
- $ea = 3$: alarma crítica.
- td : tiempo acumulado de desvío de caudal.
- tb : temporizador asociado a la condición de fin de bolsa.
- σ : tiempo de avance del modelo.

La fase *corrigiendo* representa el estado en el cual se detectó una desviación mayor al 10 % y el controlador se encuentra esperando confirmar si dicha desviación persiste para emitir una alarma media o crítica.

Si luego de emitir una **alarmaMedia** el desvío de caudal persiste durante otros 5 segundos, el controlador detiene la bomba y emite **alarmaCritica**.

8.4. Actuador de la bomba

atomic ActuadorBomba is $\langle X, Y, S, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$ where

```
S is
  d : [0,200];
  u : [0,200];
   $\sigma$  : Time;
end S

X is
  in : ({ajustarCaudal} x [0,200])  $\cup$  {detenerBomba};
end X

Y is
  out : {caudalActual} x [0,200];
end Y

 $\delta_{ext}((d,u,\sigma), e, (in,x))$  is
  defcases
    case
      u = 0;
       $\sigma$  = 0.5;
    if
      x.getFt() = detenerBomba;

    case
      u = x.getSnd();
       $\sigma$  = 0.5;
    if
      x.getFt() = ajustarCaudal;
    end defcases
  end  $\delta_{ext}$ 

 $\delta_{int}(d,u,\sigma)$  is
  d = u;
   $\sigma$  =  $\infty$ ;
end  $\delta_{int}$ 

 $\lambda(d,u,\sigma)$  is
  out = (caudalActual, u);
end  $\lambda$ 

ta(d,u, $\sigma$ ) is
   $\sigma$ ;
end ta

end atomic
```

Estado inicial: $(0, 0, \infty)$.

$$S = (d, u, \sigma)$$

donde d es el valor actual que está usando la bomba, u es el valor que llega desde el controlador y σ es la variable de tiempo de avance.

8.5. Módulo de alarmas

atomic ModuloAlarmas is $\langle X, Y, S, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$ where

S is

```
fase : {0,1,2,3};
tipo : {0,1,2,3};
 $\sigma$  : Time;
```

end S

X is

```
in : {alarmaBaja, alarmaMedia, alarmaCritica, confirmacionEnfermero};
```

end X

Y is

```
out : notificacionAlarma x {1,2,3};
```

end Y

$\delta_{ext}((fase, tipo, \sigma), e, (in, x))$ is

defcases

case

```
fase = 1;
tipo = 1;
 $\sigma$  = 0;
```

if

```
x = alarmaBaja;
```

case

```
fase = 1;
tipo = 2;
 $\sigma$  = 0;
```

if

```
x = alarmaMedia;
```

case

```
fase = 1;
tipo = 3;
 $\sigma$  = 0;
```

if

```
x = alarmaCritica;
```

case

```
fase = 0;
tipo = 0;
 $\sigma$  =  $\infty$ ;
```

if

```
x = confirmacionEnfermero;
```

end defcases

```

end  $\delta_{ext}$ 

 $\delta_{int}(fase, tipo, \sigma)$  is
defcases
  case
    fase = 0;
    tipo = 0;
     $\sigma = \infty$ ;
  if
    fase = 1 and tipo = 1;

  case
    fase = 0;
    tipo = 0;
     $\sigma = \infty$ ;
  if
    fase = 1 and tipo = 2;

  case
    fase = 2;
    tipo = 3;
     $\sigma = 30$ ;
  if
    fase = 1 and tipo = 3;

  case
    fase = 3;
    tipo = 3;
     $\sigma = 10$ ;
  if
    fase = 2 and tipo = 3;

  case
    fase = 3;
    tipo = 3;
     $\sigma = 10$ ;
  if
    fase = 3 and tipo = 3;
end defcases
end  $\delta_{int}$ 

 $\lambda(fase, tipo, \sigma)$  is
defcases
  case
    out = (notificacionAlarma, 1);
  if
    fase = 1 and tipo = 1;

  case
    out = (notificacionAlarma, 2);
  if

```



```

    fase = 1 and tipo = 2;

case
    out = (notificacionAlarma, 3);
if
    fase = 1 and tipo = 3;

case
    out = (notificacionAlarma, 3);
if
    fase = 2 and tipo = 3;

case
    out = (notificacionAlarma, 3);
if
    fase = 3 and tipo = 3;
end defcases
end λ

ta(fase,tipo,σ) is
    σ;
end ta

end atomic

```

Estado inicial: $(0, 0, \infty)$.

$$S = (fase, tipo, \sigma)$$

donde:

- $fase = 0$: pasivo.
- $fase = 1$: notificando.
- $fase = 2$: esperando confirmación.
- $fase = 3$: repitiendo.
- $tipo = 0$: ninguna.
- $tipo = 1$: baja.
- $tipo = 2$: media.
- $tipo = 3$: crítica.
- σ : tiempo de permanencia del estado.

8.6. Registro de eventos

atomic RegistroEventos is $\langle X, Y, S, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$ where

S is

```
log : List Text;
cantAlarmasBajas : N;
cantAlarmasMedias : N;
cantAlarmasCriticas : N;
cantDetenciones : N;
 $\sigma$  : Time;
```

end S

X is

```
in : {alarmaBaja, alarmaMedia, alarmaCritica, detenerBomba,
      confirmacionEnfermero, finBolsa}
     $\cup$  ({ajustarCaudal}  $\times$  [0,200]);
```

end X

Y is

```
out : {};
```

end Y

$\delta_{ext}((\log, \text{cantAlarmasBajas}, \text{cantAlarmasMedias}, \text{cantAlarmasCriticas}, \text{cantDetenciones},$
 $\sigma), e, (\text{in}, x))$ is

defcases

case

```
log = log  $\frown$  {"Alarma baja registrada"};
cantAlarmasBajas = cantAlarmasBajas + 1;
 $\sigma = \infty$ ;
```

if

```
x = alarmaBaja;
```

case

```
log = log  $\frown$  {"Alarma media registrada"};
cantAlarmasMedias = cantAlarmasMedias + 1;
 $\sigma = \infty$ ;
```

if

```
x = alarmaMedia;
```

case

```
log = log  $\frown$  {"Alarma critica registrada"};
cantAlarmasCriticas = cantAlarmasCriticas + 1;
 $\sigma = \infty$ ;
```

if

```
x = alarmaCritica;
```

case

```
log = log  $\frown$  {"Detencion de bomba registrada"};
cantDetenciones = cantDetenciones + 1;
 $\sigma = \infty$ ;
```

if

```
x = detenerBomba;
```

```

    case
        log = log  $\cup$  {"Ajuste de caudal registrado"};
         $\sigma = \infty$ ;
    if
        x.getFt() = ajustarCaudal;

    case
        log = log  $\cup$  {"Confirmacion de enfermero registrada"};
         $\sigma = \infty$ ;
    if
        x = confirmacionEnfermero;

    case
        log = log  $\cup$  {"Fin de bolsa registrado"};
         $\sigma = \infty$ ;
    if
        x = finBolsa;
    end defcases
end  $\delta_{ext}$ 

 $\delta_{int}((log, cantAlarmasBajas, cantAlarmasMedias, cantAlarmasCriticas, cantDetenciones,$ 
     $\sigma))$  is
     $\sigma = \infty$ ;
end  $\delta_{int}$ 

 $\lambda((log, cantAlarmasBajas, cantAlarmasMedias, cantAlarmasCriticas, cantDetenciones, \sigma)$ 
     $)$  is
    out = {};
end  $\lambda$ 

ta((log, cantAlarmasBajas, cantAlarmasMedias, cantAlarmasCriticas, cantDetenciones,  $\sigma$ 
    )) is
     $\sigma$ ;
end ta

end atomic

```

Estado inicial: ($\square, 0, 0, 0, 0, \infty$).

$S = (log, cantAlarmasBajas, cantAlarmasMedias, cantAlarmasCriticas, cantDetenciones, \sigma)$

donde:

- *log*: lista de textos que representa el registro histórico de eventos ocurridos.
- *cantAlarmasBajas*: cantidad de alarmas bajas registradas.
- *cantAlarmasMedias*: cantidad de alarmas medias registradas.
- *cantAlarmasCriticas*: cantidad de alarmas críticas registradas.

- *cantDetenciones*: cantidad de detenciones de bomba registradas.
- σ : tiempo de avance del modelo. Permanece en ∞ porque el registrador sólo reacciona ante eventos externos.

8.7. Bolsa

atomic Bolsa is <X, Y, S, delta_int, delta_ext, lambda, ta> where
S is

```

volumenActual : R;
caudalActual : R;
finBolsaEmitido : Bool;
sigma : Time;

```

end S

X is

```

in : {caudalActual} x [0,200];

```

end X

Y is

```

out : ({finBolsa} x {0,1}) U ({volumenBolsa} x R);

```

end Y

```

delta_ext((volumenActual, caudalActual, finBolsaEmitido, sigma), e, (in,x)) is
  caudalActual = x.getSnd();
  sigma = max(0, sigma - e);
end delta_ext

```

delta_int(volumenActual, caudalActual, finBolsaEmitido, sigma) is

defcases

case

```

  volumenActual = actualizarVolumen(caudalActual, periodo);
  finBolsaEmitido = false;
  sigma = periodo;

```

if

```

  volumenActual > umbralFinBolsa and not finBolsaEmitido;

```

case

```

  volumenActual = actualizarVolumen(caudalActual, periodo);
  finBolsaEmitido = true;
  sigma = periodo;

```

if

```

  volumenActual <= umbralFinBolsa and not finBolsaEmitido;

```

case

```

  volumenActual = actualizarVolumen(caudalActual, periodo);
  sigma = periodo;

```

if

```

  finBolsaEmitido;

```

end defcases

end delta_int

```

lambda(volumenActual, caudalActual, finBolsaEmitido, sigma) is
  defcases
  case
    out = (finBolsa, 1);
  if
    volumenActual <= umbralFinBolsa and not finBolsaEmitido;
  case
    out = (volumenBolsa, volumenActual);
  if
    volumenActual > umbralFinBolsa and not finBolsaEmitido;
  end defcases
end lambda

ta(volumenActual, caudalActual, finBolsaEmitido, sigma) is
  sigma;
end ta
end atomic

```

Estado inicial: (*volumenInicial*, 0, *false*, *periodo*)

$S = (volumenActual, caudalActual, finBolsaEmitido, \sigma)$

donde:

- *volumenActual*: volumen de medicación restante en la bolsa, en ml.
- *caudalActual*: último caudal real informado por el actuador, usado para calcular el consumo.
- *finBolsaEmitido*: indica si el evento *finBolsa* ya fue emitido, para no repetirlo en cada período.
- σ : tiempo de avance del modelo, ligado al período de actualización del volumen.

volumenInicial, *umbralFinBolsa* y *periodo* son parámetros configurables del modelo. *actualizarVolumen(caudal, Δt)* es una función auxiliar que calcula el nuevo volumen restante a partir del caudal administrado, considerando que el caudal está expresado en ml/h y el tiempo transcurrido en segundos:

$$volumenActual = volumenActual - caudal \cdot \left(\frac{\Delta t}{3600} \right)$$

8.8. Enfermero

atomic Enfermero is <X, Y, S, delta_int, delta_ext, lambda, ta> where

S is

```

  esperandoRespuesta : Bool;
  sigma : Time;
end S

```

X is

```

    in : {notificacionAlarma} x {1,2,3};
end X

Y is
    out : {confirmacionEnfermero} x {1};
end Y

delta_ext((esperandoRespuesta, sigma), e, (in,x)) is
    defcases
    case
        esperandoRespuesta = true;
        sigma = tiempoRespuesta();
    if
        x.getFt() = notificacionAlarma and not esperandoRespuesta;
    case
        esperandoRespuesta = esperandoRespuesta;
        sigma = max(0, sigma - e);
    if
        x.getFt() = notificacionAlarma and esperandoRespuesta;
    end defcases
    end delta_ext

delta_int(esperandoRespuesta, sigma) is
    esperandoRespuesta = false;
    sigma = infinity;
end delta_int

lambda(esperandoRespuesta, sigma) is
    out = (confirmacionEnfermero, 1);
end lambda

ta(esperandoRespuesta, sigma) is
    sigma;
end ta
end atomic

```

Estado inicial: (*false*, ∞)

$S = (esperandoRespuesta, \sigma)$

donde:

- *esperandoRespuesta* = *false*: el enfermero está a la espera de una nueva notificación de alarma.
- *esperandoRespuesta* = *true*: el enfermero ya recibió una notificación y está dentro del intervalo de tiempo de respuesta antes de confirmar.
- σ : tiempo restante hasta emitir la *confirmacionEnfermero*.

tiempoRespuesta() es una función que devuelve el tiempo de respuesta del enfermero según el modo configurado (determinístico, uniforme o exponencial), utilizando los parámetros *tiempoFijo*, *minTiempo*, *maxTiempo* o *lambdaTiempo*.

9. Modelo acoplado

El modelo acoplado representa la integración de los modelos atómicos definidos anteriormente. En este sistema, el **Controlador de bomba** actúa como componente central, recibiendo las órdenes médicas, las mediciones del sensor de flujo, la señal de fin de bolsa y las confirmaciones del enfermero.

A partir de estos eventos, el controlador decide si debe ajustar el caudal, detener la bomba o emitir alarmas. El **Actuador de la bomba** modifica el caudal real administrado, mientras que el **Sensor de flujo** mide periódicamente dicho caudal y lo informa nuevamente al controlador.

Además, el **Módulo de alarmas** gestiona las notificaciones generadas y el **Registro de eventos** almacena los eventos relevantes del sistema.

9.1. Especificación formal del acoplamiento

El modelo acoplado se define como:

$$N = \langle X_N, Y_N, D, \{M_d\}, EIC, EOC, IC, Select \rangle$$

donde N representa el sistema completo de bomba de infusión simplificada.

$$D = \{ \begin{array}{l} \textit{ControladorBomba}, \\ \textit{SensorFlujo}, \\ \textit{ActuadorBomba}, \\ \textit{ModuloAlarmas}, \\ \textit{RegistroEventos}, \\ \textit{Bolsa}, \\ \textit{Enfermero} \end{array} \}$$

Los modelos contenidos en el conjunto D corresponden a los componentes internos del sistema acoplado. El *Generador de Órdenes Médicas* se considera parte del entorno de simulación y no pertenece al modelo acoplado, por lo que su salida ingresa a N como una entrada externa.

Entradas y salidas externas

La entrada externa formal del sistema es:

$$X_N = \{pOrdenMedica\}$$

La salida externa formal del sistema es:

$$Y_N = \{pNotificacionAlarma\}$$

El componente *RegistroEventos* no se incluye como salida externa formal del modelo acoplado, ya que su función es mantener internamente un registro histórico o *log* de los eventos ocurridos durante la simulación. Dicho registro puede consultarse luego para el análisis de resultados, pero no se modela como un evento de salida DEVS.

Acoplamiento de entrada externa

Los acoplamientos de entrada externa, EIC , conectan las entradas del modelo acoplado N con los componentes internos correspondientes:

$$EIC = \{[(N, pOrdenMedica), (ControladorBomba, pOrdenMedica)]\}$$

La orden médica es generada por el entorno de simulación mediante el modelo Generador de Órdenes Médicas, el cual no forma parte del conjunto de componentes internos del modelo acoplado. Por este motivo, la orden médica ingresa al sistema a través del puerto externo $pOrdenMedica$ y es enviada al controlador para que determine las acciones correspondientes.

Acoplamiento de salida externa

Los acoplamientos de salida externa, EOC , conectan las salidas de los modelos internos con las salidas externas del sistema acoplado:

$$EOC = \{[(ModuloAlarmas, pNotificacionAlarma), (N, pNotificacionAlarma)]\}$$

La salida externa del sistema corresponde a las notificaciones emitidas por el módulo de alarmas.

Acoplamiento interno

Los acoplamientos internos, IC , conectan las salidas de los modelos atómicos con las entradas de otros modelos atómicos dentro del sistema acoplado:

$$IC = \{$$

$$[(ControladorBomba, pAjustarCaudal), (ActuadorBomba, pAjustarCaudal)];$$

$$[(ControladorBomba, pDetenerBomba), (ActuadorBomba, pDetenerBomba)];$$

$$[(ControladorBomba, pAlarmaBaja), (ModuloAlarmas, pAlarmaBaja)];$$

$$[(ControladorBomba, pAlarmaMedia), (ModuloAlarmas, pAlarmaMedia)];$$

$$[(ControladorBomba, pAlarmaCritica), (ModuloAlarmas, pAlarmaCritica)];$$

$$[(ActuadorBomba, pCaudalActual), (SensorFlujo, pCaudalActual)];$$

$[(SensorFlujo, pSensorFlujo), (ControladorBomba, pSensorFlujo)];$

$[(ActuadorBomba, pCaudalActual), (Bolsa, pCaudalActual)];$

$[(Bolsa, pFinBolsa), (ControladorBomba, pFinBolsa)];$

$[(Bolsa, pFinBolsa), (RegistroEventos, pFinBolsa)];$

$[(ModuloAlarmas, pNotificacionAlarma), (Enfermero, pNotificacionAlarma)];$

$[(Enfermero, pConfirmacionEnfermero), (ControladorBomba, pConfirmacionEnfermero)];$

$[(Enfermero, pConfirmacionEnfermero), (ModuloAlarmas, pConfirmacionEnfermero)];$

$[(Enfermero, pConfirmacionEnfermero), (RegistroEventos, pConfirmacionEnfermero)];$

$[(ControladorBomba, pAjustarCaudal), (RegistroEventos, pAjustarCaudal)];$

$[(ControladorBomba, pDetenerBomba), (RegistroEventos, pDetenerBomba)];$

$[(ControladorBomba, pAlarmaBaja), (RegistroEventos, pAlarmaBaja)];$

$[(ControladorBomba, pAlarmaMedia), (RegistroEventos, pAlarmaMedia)];$

$[(ControladorBomba, pAlarmaCritica), (RegistroEventos, pAlarmaCritica)]\}$

El controlador recibe las mediciones periódicas del sensor de flujo y, a partir de ellas, puede emitir órdenes de ajuste de caudal o detención de la bomba. El actuador modifica el caudal real administrado y comunica dicho caudal al sensor de flujo. Luego, el sensor envía la medición al controlador.

Las alarmas generadas por el controlador son enviadas al módulo de alarmas para su gestión. Además, las acciones de ajuste de caudal, detención de bomba y emisión de alarmas son enviadas al modelo *RegistroEventos*, encargado de mantener el historial interno de funcionamiento del sistema.

Función de desempate

Para resolver posibles eventos simultáneos entre componentes, se define la siguiente prioridad:

$Select = \langle ControladorBomba, ActuadorBomba, SensorFlujo, ModuloAlarmas, RegistroEventos, Bol$

Esta prioridad favorece primero la toma de decisiones del controlador, luego la ejecución física de las acciones mediante el actuador, la actualización de las mediciones del sensor, la gestión de alarmas y finalmente el registro de eventos.

10. Implementación en PowerDEVS

Para la simulación del sistema se utilizó PowerDEVS. Cada uno de los modelos atómicos definidos en la especificación formal fue implementado en C++, respetando las entradas, salidas, estados y funciones de transición establecidas en el modelo DEVS.

La implementación reproduce la estructura del modelo acoplado presentado anteriormente. El sistema está compuesto por un controlador central, un actuador de bomba, un sensor de flujo, un módulo de alarmas y un registro de eventos. Además, durante el desarrollo de la implementación fue necesario incorporar dos modelos adicionales: un modelo de enfermero y un modelo de bolsa de medicación. Estos módulos permitieron simular escenarios de confirmación de alarmas y agotamiento progresivo de la bolsa.

10.1. Estructura general del modelo

La estructura general del modelo implementado en PowerDEVS representa el acoplamiento entre los distintos componentes del sistema. El controlador recibe las órdenes médicas, las mediciones del sensor, las señales de fin de bolsa y las confirmaciones del enfermero. A partir de esta información, decide si debe ajustar el caudal, detener la bomba o emitir una alarma.

El actuador modifica el caudal administrado, el sensor mide periódicamente dicho caudal, el módulo de alarmas gestiona las notificaciones generadas y el registro de eventos almacena los sucesos relevantes de la simulación.

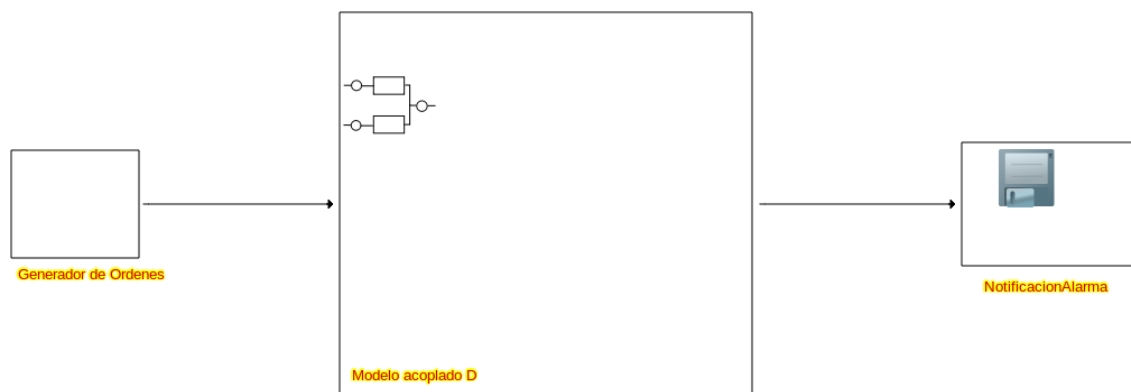


Figura 2: Modelo completo en PowerDEVS.

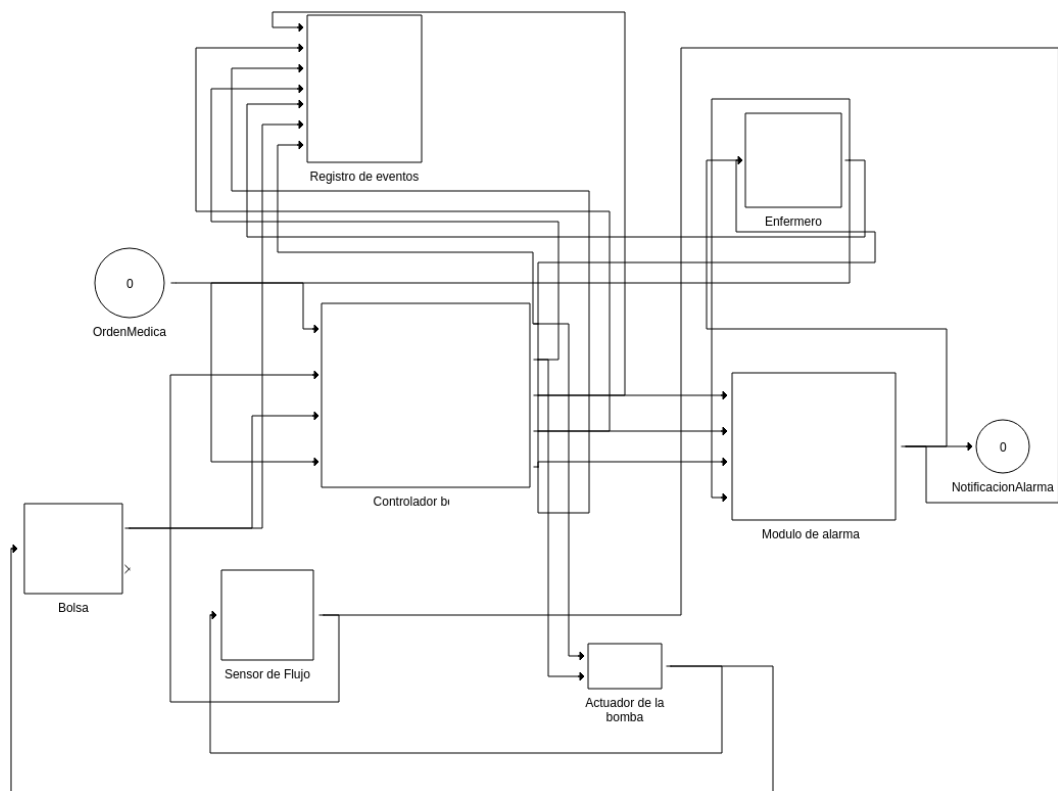


Figura 3: Modelo acoplado en PowerDEVS.

Módulo: Generador de Órdenes Médicas

- **Función:** Genera eventos `ordenMedica` que indican el caudal objetivo de infusión. Este módulo permite definir los distintos escenarios de simulación, por ejemplo iniciar la bomba con un caudal determinado, cambiar la orden médica durante la infusión o enviar una orden con caudal igual a cero.
- **Entradas:**
 - Ninguna.
- **Salidas:**
 - `ordenMedica`.

Módulo: Controlador de Bomba

- **Función:** Supervisa el comportamiento general del sistema y toma las decisiones de control. Recibe las órdenes médicas, las mediciones del sensor de flujo, la señal de fin de bolsa y las confirmaciones del enfermero. A partir de esta información, decide si debe iniciar la infusión, modificar el caudal, detener la bomba o emitir alarmas.
- **Entradas:**

- ordenMedica.
- sensorFlujo.
- finBolsa.
- confirmacionEnfermero.

■ **Salidas:**

- ajustarCaudal.
- detenerBomba.
- alarmaBaja.
- alarmaMedia.
- alarmaCritica.

Módulo: Actuador de la Bomba

- **Función:** Representa el mecanismo físico encargado de administrar la medicación. Recibe órdenes del controlador y modifica el caudal real administrado por la bomba. También se encarga de llevar el caudal a cero cuando recibe la orden de detención.

■ **Entradas:**

- ajustarCaudal.
- detenerBomba.

■ **Salidas:**

- caudalActual.

Módulo: Sensor de Flujo

- **Función:** Mide periódicamente el caudal administrado por la bomba y lo informa al controlador. En la implementación se incorporó un modo de falla que permite modificar la medición mediante un factor multiplicativo, con el objetivo de simular desvíos entre el caudal real y el caudal informado.

■ **Entradas:**

- caudalActual.

■ **Salidas:**

- sensorFlujo.

- **Consideración de diseño:** El modo de falla del sensor fue incorporado para poder evaluar escenarios en los cuales el caudal informado difiere del caudal esperado. Esto permite verificar la detección de desvíos, la emisión de alarmas medias y el escalamiento a alarma crítica.

Módulo: Módulo de Alarmas

- **Función:** Gestiona las alarmas generadas por el controlador. Puede recibir alarmas bajas, medias o críticas y emitir una notificación de alarma. En el caso de alarmas críticas, el modelo permite representar la espera de confirmación y la repetición periódica cuando la alarma no es reconocida.
- **Entradas:**
 - alarmaBaja.
 - alarmaMedia.
 - alarmaCritica.
 - confirmacionEnfermero.
- **Salidas:**
 - notificacionAlarma.

Módulo: Registro de Eventos

- **Función:** Almacena los eventos relevantes producidos durante la simulación. Este módulo permite registrar alarmas, ajustes de caudal, detenciones, confirmaciones y eventos de fin de bolsa para su posterior análisis.
- **Entradas:**
 - alarmaBaja.
 - alarmaMedia.
 - alarmaCritica.
 - ajustarCaudal.
 - detenerBomba.
 - confirmacionEnfermero.
 - finBolsa.
- **Salidas:**
 - Ninguna.

Durante el desarrollo de la implementación en PowerDEVS se observó que, además de los modelos definidos inicialmente, era necesario incorporar dos nuevos módulos para representar con mayor realismo algunos de los escenarios pedidos en la consigna. Estos modelos son el módulo **Enfermero** y el módulo **Bolsa**.

El módulo **Enfermero** permite simular la confirmación de una alarma por parte del personal médico, mientras que el módulo **Bolsa** permite representar el consumo progresivo del volumen disponible y la emisión automática del evento **finBolsa**. De esta manera, fue posible simular escenarios como fin de bolsa con confirmación del enfermero y alarma crítica no confirmada durante 30 segundos.

Módulo: Enfermero

- **Función:** Simula la respuesta del personal médico ante una notificación de alarma. Cuando recibe una notificación, el modelo espera un determinado tiempo de respuesta y luego emite una `confirmacionEnfermero`.
- **Entradas:**
 - `notificacionAlarma`.
- **Salidas:**
 - `confirmacionEnfermero`.
- **Consideración de diseño:** Este modelo se incorporó para poder evaluar el comportamiento del sistema cuando una alarma es confirmada o cuando la confirmación se retrasa. También permite analizar la propiedad temporal que indica que una alarma crítica no confirmada debe repetirse luego de 30 segundos y posteriormente cada 10 segundos.

Módulo: Bolsa

- **Función:** Representa la bolsa de medicación y calcula el volumen restante a partir del caudal administrado por la bomba. Cuando el volumen disponible alcanza un umbral mínimo, emite el evento `finBolsa`.
- **Entradas:**
 - `caudalActual`.
- **Salidas:**
 - `finBolsa`.
 - `volumenBolsa`.
- **Consideración de diseño:** Este modelo se incorporó para que el evento `finBolsa` no sea generado manualmente, sino como consecuencia del consumo de medicación durante la simulación. El consumo se calcula considerando que el caudal se encuentra expresado en ml/h y el tiempo transcurrido en segundos.

10.2. Organización de los archivos

Todos los módulos descritos corresponden a implementaciones desarrolladas en PowerDEVS utilizando C++.

Para que PowerDEVS pueda reconocer correctamente los modelos atómicos implementados, los archivos fuente `.cpp` y los archivos de cabecera `.h` o `.hpp` deben ubicarse dentro del directorio:

`/PowerDEVS/atomics/bomba/`

La carpeta `bomba` no forma parte de la instalación estándar de PowerDEVS, por lo tanto debe ser creada manualmente antes de copiar los archivos correspondientes. Una vez ubicados los modelos en dicha ruta, estos pueden incorporarse al entorno de simulación y utilizarse como bloques atómicos dentro de los modelos acoplados.

10.3. Parametrización del sistema

La parametrización facilita la realización de distintos experimentos de simulación. En la implementación desarrollada, los parámetros configurables permiten modificar los tiempos de generación de órdenes, el comportamiento del sensor, la respuesta del enfermero, el volumen de la bolsa y la duración total de la simulación.

Tiempo máximo de simulación

El tiempo máximo de simulación se configura desde la interfaz principal de PowerDEVS mediante el campo **Final Time**. Este valor determina la duración total de la ejecución.

Para pruebas muy simples puede utilizarse un valor bajo, como 10 segundos. Sin embargo, para los escenarios pedidos en la consigna, ese tiempo no siempre resulta suficiente, ya que algunas propiedades temporales requieren observar eventos posteriores a los 30 o 60 segundos. Por ejemplo, la repetición de una alarma crítica recién puede verificarse luego de 30 segundos sin confirmación, y la detención por fin de bolsa debe analizarse hasta 60 segundos después de detectado el evento `finBolsa`.

Escenario	Final Time sugerido
Funcionamiento normal	20 s
Cambio de orden médica	40 s
Orden médica con caudal cero	40 s
Desvío leve corregido	20 s
Desvío mayor al 10 % durante más de 5 s	20 s
Fin de bolsa	80 s o más
Alarma crítica no confirmada	70 s o más

Cuadro 2: Tiempos máximos de simulación sugeridos para cada escenario.

Para el escenario de fin de bolsa, el tiempo necesario depende del volumen inicial, del umbral definido y del caudal administrado. Si se utiliza una bolsa de 200 ml con umbral de 10 ml y un caudal de 100 ml/h, el evento `finBolsa` ocurrirá recién luego de varias horas simuladas. Por este motivo, para pruebas de validación se recomienda configurar el volumen inicial cercano al umbral, de modo que el evento `finBolsa` pueda observarse dentro de una simulación corta.

Parámetros de los módulos

Generador de Órdenes Médicas

- **modo**: define el tipo de tiempo entre órdenes. Valor 0: determinístico, valor 1: uniforme y valor 2: exponencial.
- **tiempoFijo**: tiempo utilizado en modo determinístico. Valor inicial: 100 s.
- **minTiempo**: tiempo mínimo utilizado en modo uniforme. Valor inicial: 5 s.
- **maxTiempo**: tiempo máximo utilizado en modo uniforme. Valor inicial: 15 s.
- **lambdaTiempo**: parámetro utilizado para generar tiempos con distribución exponencial. Valor inicial: 0.1.

Sensor de Flujo

- Período de muestreo: 1 s.
- `modoFalla`: habilita o deshabilita una falla de medición.
- `factorFalla`: factor utilizado para modificar la medición cuando el modo de falla está activo.

Enfermero

- `modo`: define el tipo de tiempo de respuesta. Valor 0: determinístico, valor 1: uniforme y valor 2: exponencial.
- `tiempoFijo`: tiempo de respuesta utilizado en modo determinístico. Valor inicial: 15 s.
- `minTiempo`: tiempo mínimo de respuesta en modo uniforme. Valor inicial: 5 s.
- `maxTiempo`: tiempo máximo de respuesta en modo uniforme. Valor inicial: 20 s.
- `lambdaTiempo`: parámetro utilizado para generar tiempos de respuesta con distribución exponencial. Valor inicial: 0.1.

Bolsa

- `volumenInicial`: volumen inicial de medicación disponible. Valor inicial: 200 ml.
- `umbralFinBolsa`: volumen mínimo a partir del cual se emite el evento `finBolsa`. Valor inicial: 10 ml.
- `periodo`: intervalo de actualización del volumen restante. Valor inicial: 1 s.

11. Resultados de Simulación y Análisis

Finalizada la etapa de implementación en PowerDEVS, se procedió a la ejecución de distintos escenarios de simulación con el objetivo de verificar el correcto funcionamiento del sistema y validar los requisitos temporales, de seguridad y de vivacidad establecidos en la consigna.

Los resultados obtenidos se analizaron a partir de los registros generados por el módulo `RegistroEventos` y de las señales almacenadas mediante los bloques `To Disk` de PowerDEVS. Estos datos permitieron construir gráficos y evaluar el comportamiento del sistema ante condiciones normales y situaciones de falla.

En las siguientes secciones se presentan los resultados obtenidos para cada escenario de simulación, junto con el análisis correspondiente y la verificación de las propiedades especificadas para el sistema.

11.1. Funcionamiento Normal sin fallas

En este escenario se simuló el funcionamiento normal del sistema, sin fallas en el sensor, sin eventos de fin de bolsa y sin intervención del enfermero. La orden médica inicial indicó un caudal objetivo de 100 ml/h.

El objetivo fue verificar que la bomba iniciara la infusión dentro del tiempo requerido y que el caudal real se mantuviera igual al caudal indicado, sin generar alarmas.

- Caudal objetivo: 100 ml/h.
- Modo de falla del sensor: desactivado.
- Tiempo final de simulación: 60 s.
- Modo de generador de orden medica: deterministico

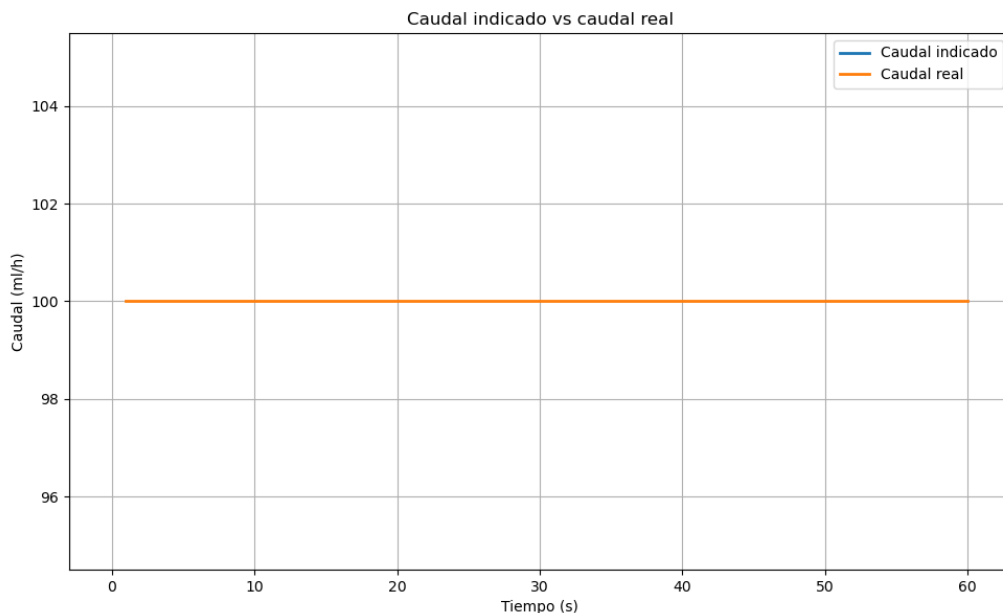


Figura 4: Caudal indicado versus caudal real durante el funcionamiento normal del sistema.

Como se observa en la Figura 4, el caudal real coincide con el caudal indicado durante toda la simulación. Este resultado confirma que el sistema cumple con el comportamiento esperado en ausencia de fallas y que la bomba inicia y mantiene la infusión de manera correcta.

Además, durante la simulación no se registraron alarmas ni detenciones preventivas, verificándose el cumplimiento de los requisitos de funcionamiento normal definidos para el sistema.

11.2. Cambio de orden médica durante la infusión

En este escenario se simuló un funcionamiento normal del sistema en el que se producen cambios en las órdenes médicas mientras la infusión se encuentra en curso. Dichas órdenes

son recibidas por el controlador, que debe ajustar el caudal administrado de acuerdo con los nuevos valores indicados.

El objetivo fue verificar que la bomba iniciara la infusión dentro del tiempo requerido y que respondiera correctamente ante los cambios de orden médica, manteniendo el caudal real lo más cercano posible al caudal indicado.

- Modo de falla del sensor: desactivado.
- Tiempo final de simulación: 60 s.
- Modo del generador de órdenes médicas: determinístico.

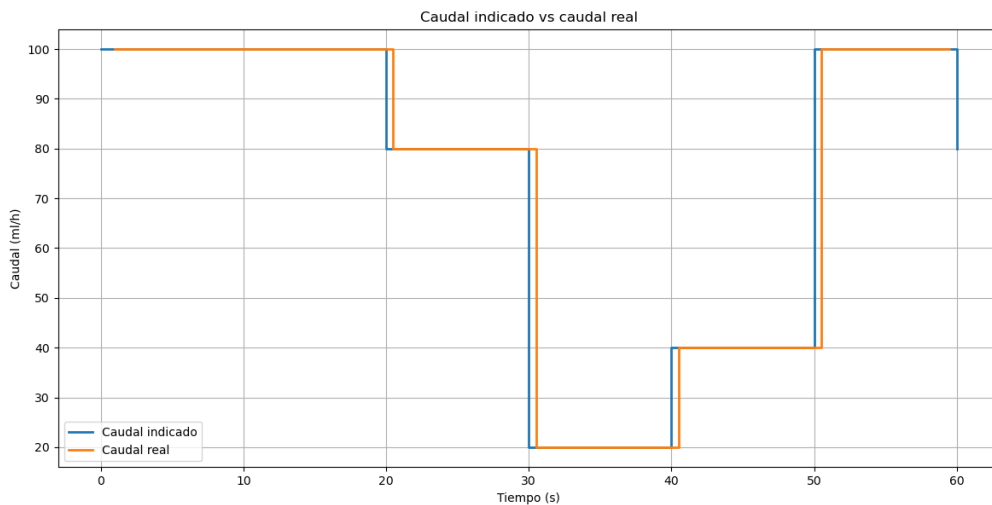


Figura 5: Caudal indicado versus caudal real durante cambios sucesivos de órdenes médicas.

Como se observa en la Figura 5, el caudal real sigue correctamente los cambios realizados sobre el caudal indicado. Cada vez que el controlador recibe una nueva orden médica, el actuador ajusta el caudal administrado hasta alcanzar el nuevo valor solicitado.

Durante la simulación no se registraron alarmas ni detenciones, lo que indica que los cambios de referencia fueron procesados correctamente por el sistema. Este escenario permitió verificar que toda orden médica recibida produce una acción sobre la bomba y que el caudal administrado se mantiene dentro de los límites establecidos.

11.3. Orden médica con caudal igual a cero

En este escenario se verificó el comportamiento del sistema cuando la orden médica indica un caudal igual a cero. Para ello, se configuró el generador de órdenes médicas para emitir inicialmente una orden con caudal objetivo nulo.

El objetivo fue comprobar que la bomba no administrara medicación ante una orden médica de valor cero y que el controlador emitiera la correspondiente orden de detención.

- Caudal objetivo inicial: 0 ml/h.
- Modo de falla del sensor: desactivado.

- Tiempo final de simulación: 60 s.
- Modo del generador de órdenes médicas: determinístico.

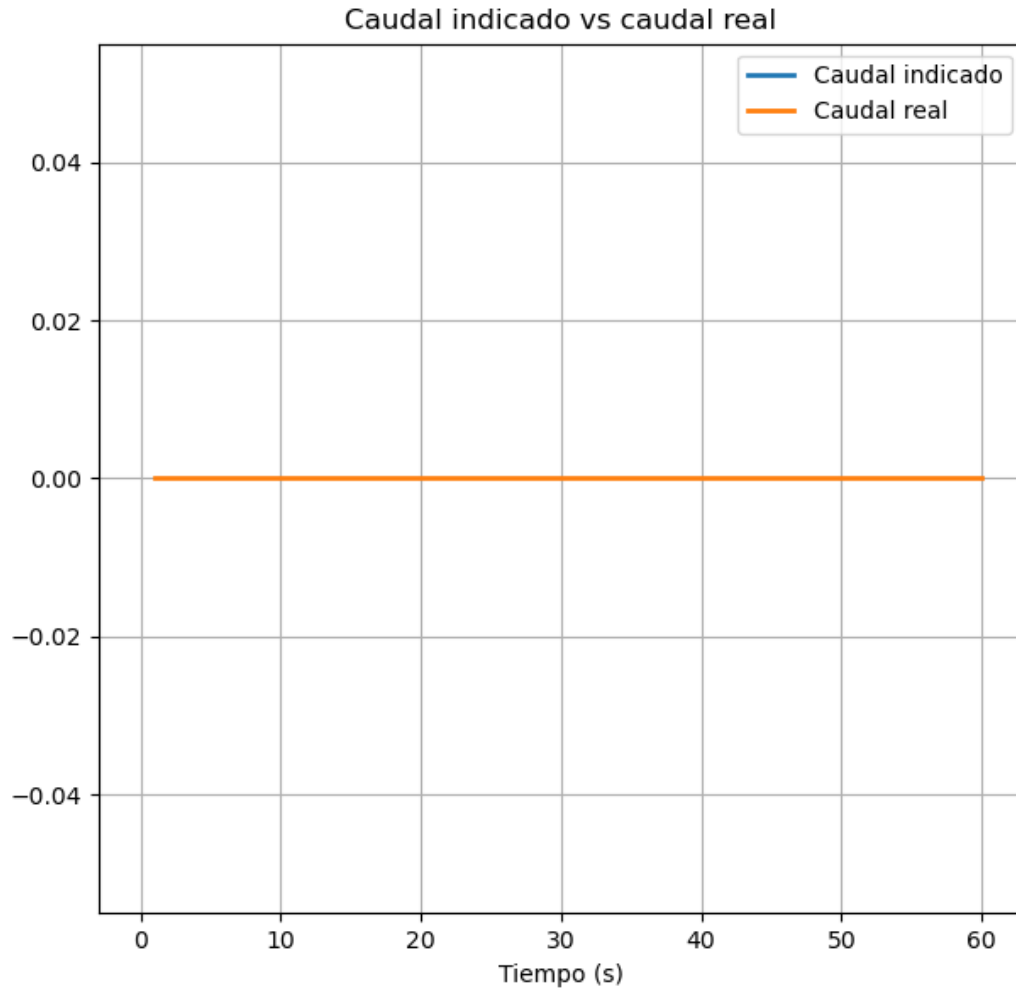


Figura 6: Caudal indicado versus caudal real para una orden médica con caudal igual a cero.

Ademas, el registro de eventos obtenido durante la simulación fue el siguiente:

```
Registro de eventos
t=0 Detencion de bomba registrada
```

```
--- RESUMEN ---
Alarmas bajas: 0
Alarmas medias: 0
Alarmas criticas: 0
Detenciones: 1
```

Como puede observarse en la Figura 6, el controlador emitió una única orden de detención al inicio de la simulación y no se registraron alarmas. Esto indica que la bomba

permaneció detenida durante toda la ejecución, cumpliendo el requisito de no administrar medicación cuando la última orden médica recibida indica un caudal igual a cero.

Este escenario permitió verificar la propiedad de seguridad que establece que la bomba no debe administrar medicación ante una orden médica nula.

11.4. Desvío leve de caudal

En este escenario se simuló un desvío leve en la medición del caudal mediante la activación del modo de falla del sensor. Se configuró un factor de falla de 0.95, provocando que el controlador recibiera una medición un 5 % menor que el caudal real administrado.

El objetivo fue verificar que el sistema continuara operando normalmente ante desviaciones menores al umbral establecido y que no se generaran alarmas innecesarias.

- Caudal objetivo: 100 ml/h.
- Modo de falla del sensor: activado.
- Factor de falla: 0.95.
- Desvío introducido: 5 %.
- Tiempo final de simulación: 60 s.

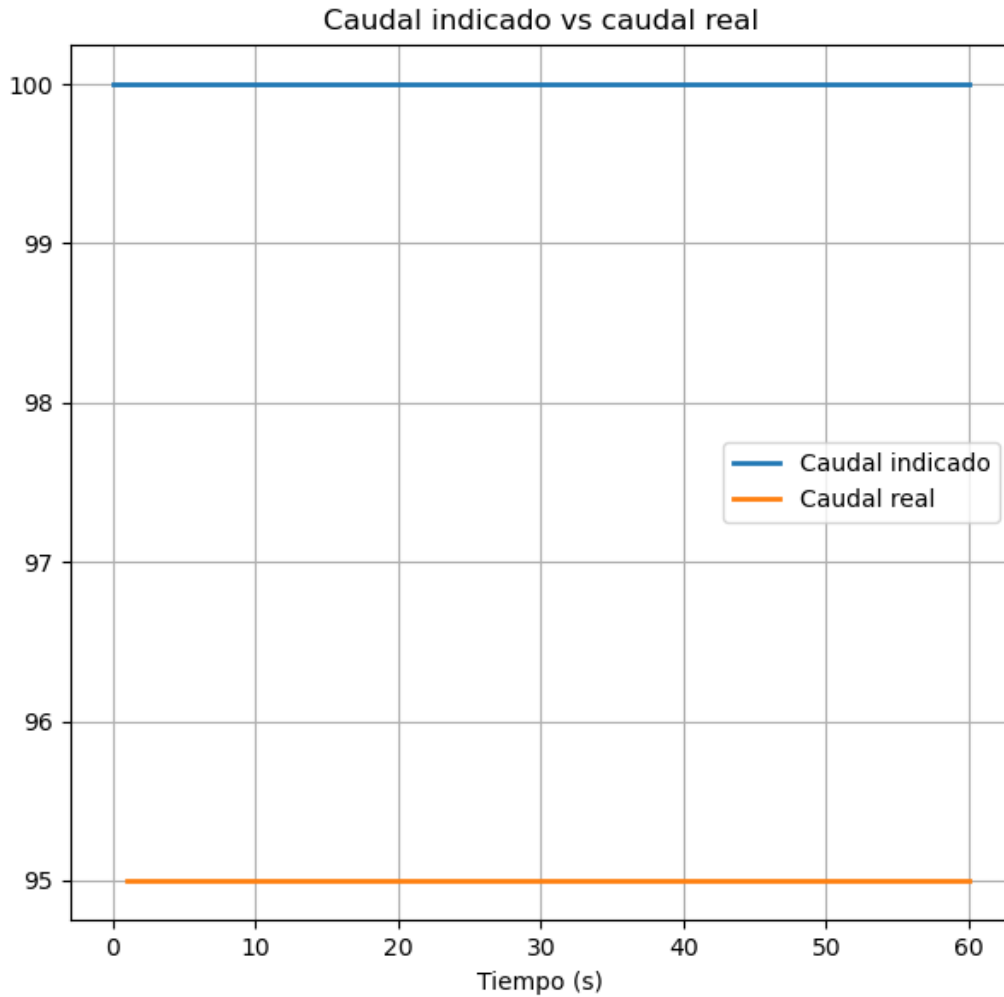


Figura 7: Caudal indicado versus caudal real ante un desvío leve de medición.

Como se observa en la Figura 7, el caudal indicado permanece en 100 ml/h mientras que el caudal medido se mantiene en 95 ml/h. La diferencia corresponde a un desvío del 5 %, valor inferior al umbral del 10 % definido para la detección de fallas.

Durante la simulación no se registraron alarmas ni detenciones. El registro de eventos únicamente mostró el ajuste inicial de caudal, lo que indica que el sistema continuó operando normalmente ante una desviación considerada aceptable por la especificación.

11.5. Desvío de caudal mayor al 10 %

En este escenario se simuló una falla en la medición del caudal mediante el modo de falla del sensor. Se configuró un factor de falla de 0.8, por lo que ante un caudal objetivo de 100 ml/h el caudal informado al controlador fue de 80 ml/h.

El objetivo fue verificar que, ante un desvío mayor al 10 % sostenido durante más de 5 segundos, el sistema emitiera una **alarmaMedia**. Además, se observó el comportamiento posterior del controlador cuando el desvío persistió luego de dicha alarma.

- Caudal objetivo: 100 ml/h.
- Modo de falla del sensor: activado.

- Factor de falla: 0.8.
- Desvío introducido: 20 %.
- Tiempo final de simulación: 60 s.

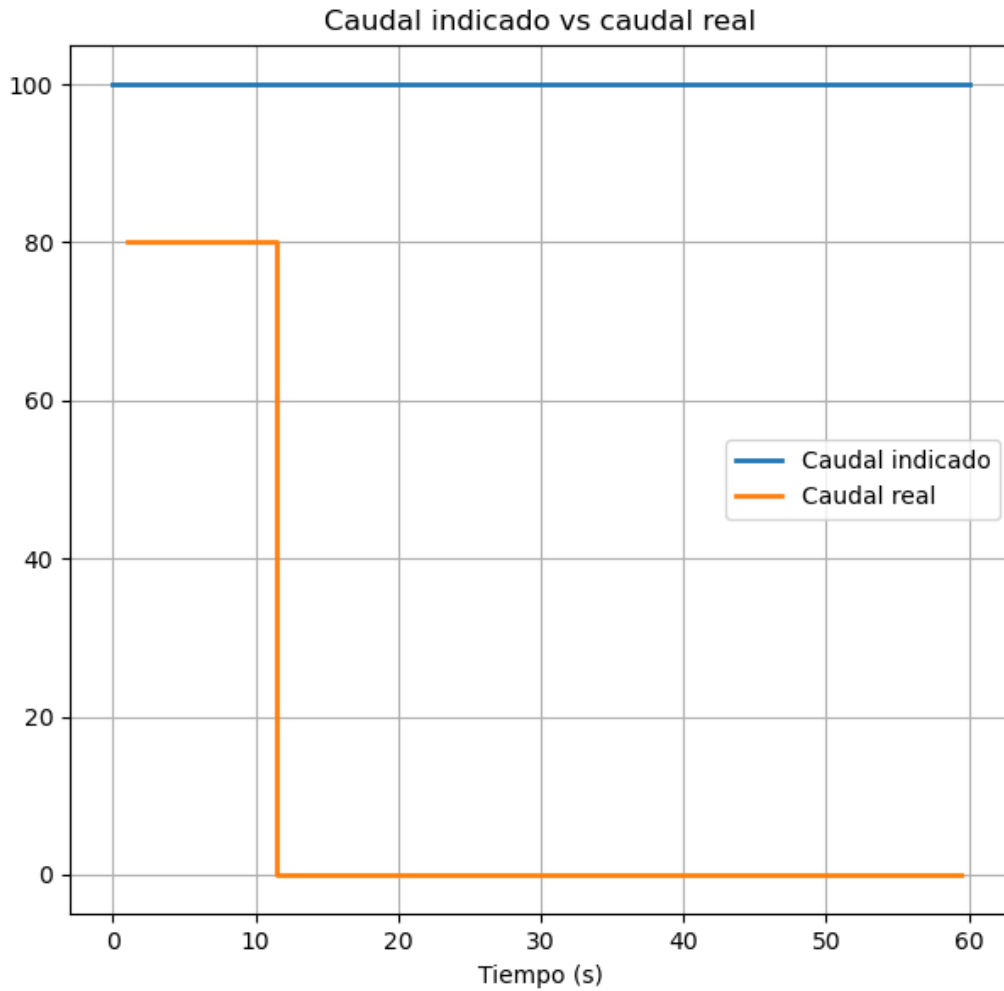


Figura 8: Caudal indicado versus caudal real ante un desvío mayor al 10 %.

Como se observa en la Figura 8, el caudal real informado permanece en 80 ml/h mientras que el caudal indicado es de 100 ml/h. Esta diferencia representa un desvío del 20 %, superior al umbral permitido del 10 %.

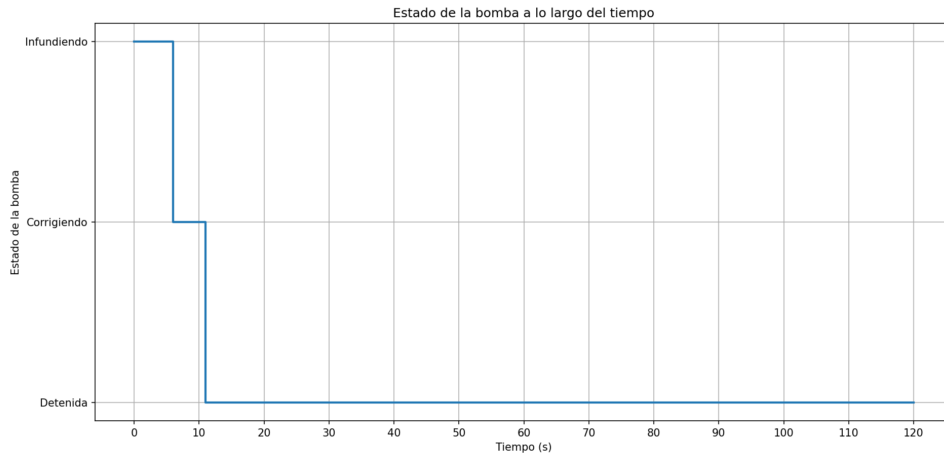


Figura 9: Evolución del estado de la bomba ante un desvío persistente de caudal.

Como se observa en la Figura 9, la bomba inicia en estado de infusión normal. Al detectarse un desvío de caudal superior al 10 %, el controlador pasa al estado de corrección. Luego de mantenerse el desvío durante más de 5 segundos, se genera una alarma crítica y la bomba se detiene preventivamente.

El registro de eventos obtenido fue el siguiente:

```
Registro de eventos
t=0 Ajuste de caudal registrado: 100
t=6 Alarma media registrada
t=11 Alarma critica registrada
t=11 Detencion de bomba registrada
t=16 Confirmacion de enfermero registrada
```

```
--- RESUMEN ---
Alarmas bajas: 0
Alarmas medias: 1
Alarmas criticas: 1
Detenciones: 1
```

Además, el archivo de notificaciones de alarma registró los siguientes eventos:

```
6, 2
11, 3
```

donde el valor 2 corresponde a una alarma media y el valor 3 a una alarma crítica.

Estos resultados verifican que el sistema emite una `alarmaMedia` cuando el desvío de caudal supera el 10 % durante más de 5 segundos. También se observa que, al persistir el desvío, el controlador escala la situación a `alarmaCritica` y detiene preventivamente la bomba.

11.6. Fin de bolsa con confirmación del enfermero

En este escenario se simuló el agotamiento progresivo de la bolsa de medicación durante la infusión. Para ello, se utilizó el módulo `Bolsa`, encargado de actualizar el volumen

restante a partir del caudal administrado. Cuando el volumen alcanzó el umbral mínimo configurado, se generó el evento `finBolsa`.

El objetivo fue verificar que el sistema emitiera una alarma baja ante la detección de fin de bolsa, que dicha alarma pudiera ser confirmada por el enfermero y que la bomba se detuviera automáticamente dentro del tiempo máximo permitido.

- Caudal objetivo: 100 ml/h.
- Volumen inicial de la bolsa: 11 ml.
- Umbral de fin de bolsa: 10 ml.
- Período de actualización de la bolsa: 1 s.
- Tiempo de respuesta del enfermero: 5 s.
- Tiempo final de simulación: 120 s.

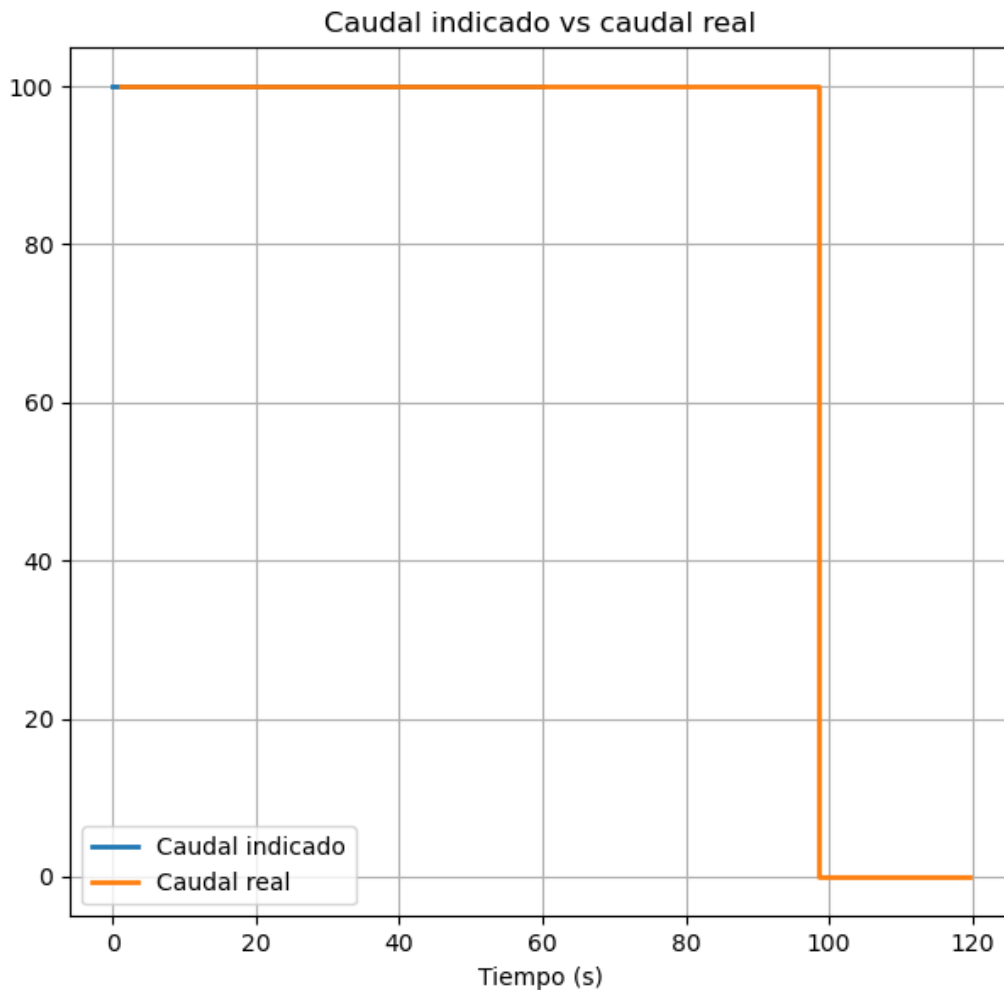


Figura 10: Caudal indicado versus caudal real ante la detección de fin de bolsa.

Como se observa en la Figura 10, la bomba mantiene el caudal indicado durante la primera parte de la simulación. Luego de detectarse el evento `finBolsa`, el sistema permite

continuar la infusión durante un intervalo limitado y posteriormente detiene la bomba de manera automática. El controlador emitió la orden de detención en $t = 97.5$ s, y el caudal real, debido a la latencia del actuador, alcanzó 0 ml/h en $t = 98.0$ s, tal como se observa en la caída de la curva naranja hacia el final del gráfico.

El registro de eventos obtenido fue el siguiente:

```
Registro de eventos
t=0 Ajuste de caudal registrado: 100
t=37.5 Fin de bolsa registrado
t=37.5 Alarma baja registrada
t=97.5 Detencion de bomba registrada
```

```
--- RESUMEN ---
Alarmas bajas: 1
Alarmas medias: 0
Alarmas criticas: 0
Detenciones: 1
```

Además, el archivo de notificaciones de alarma registró el evento:

- $t = 37,5$ s: alarma baja.

11.7. Alarma crítica no confirmada durante 30 segundos

En este escenario se simuló una falla persistente del sensor de flujo que provocó un desvío superior al 10 % respecto del caudal indicado. Luego de emitirse una alarma crítica, no se envió ninguna confirmación del enfermero, con el objetivo de verificar el comportamiento del sistema ante una alarma no atendida.

- Caudal objetivo: 100 ml/h.
- Sensor en modo falla.
- Factor de falla: 0.8.
- Confirmación del enfermero: deshabilitada.
- Tiempo final de simulación: 120 s.

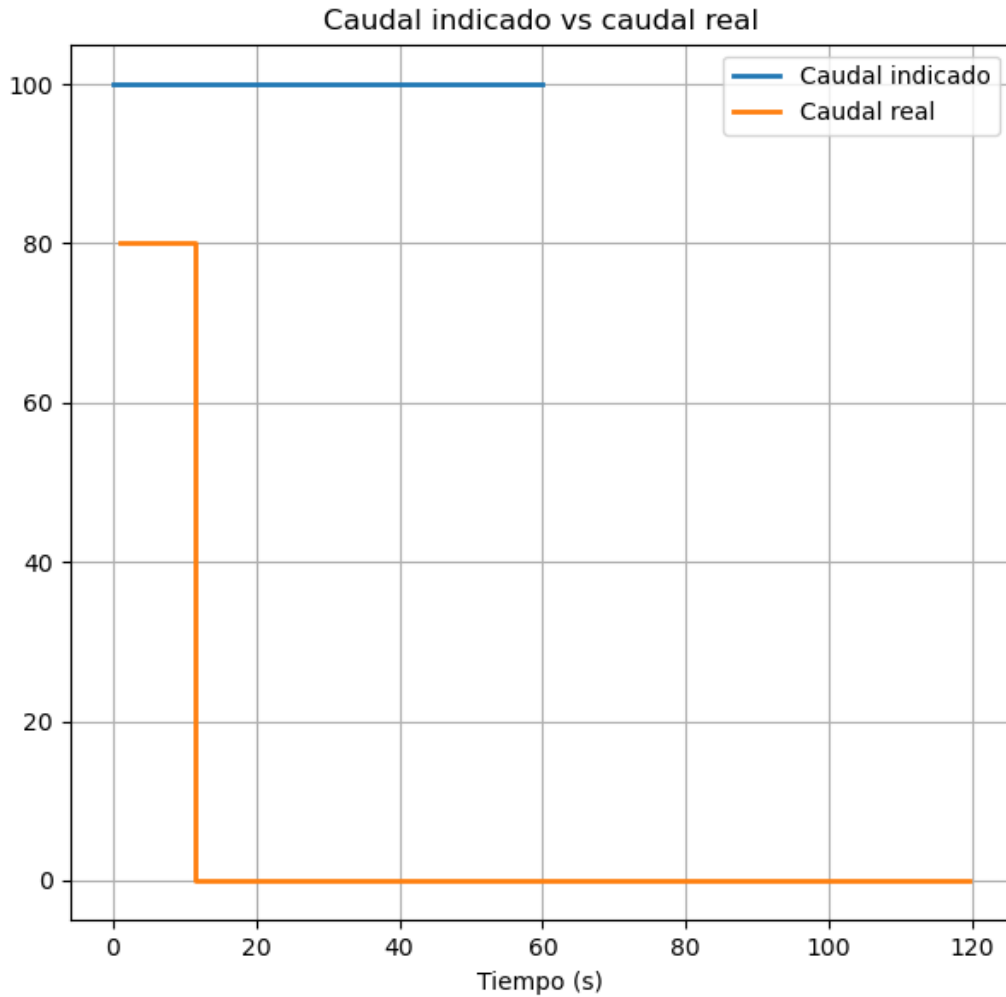


Figura 11: Caudal indicado versus caudal real durante una condición de alarma crítica no confirmada.

Como se observa en la Figura 11, el caudal real presenta un desvío permanente respecto del valor indicado. A los 6 segundos se genera una alarma media y a los 11 segundos una alarma crítica, provocando la detención de la bomba. Al no recibirse confirmación del enfermero, el módulo de alarmas repite periódicamente la alarma crítica, cumpliendo con el requisito temporal especificado.

Ninguno de los resultados muestran una violación de propiedades. Si bien el escenario introduce una condición de falla o funcionamiento anómalo, el sistema responde de acuerdo con la especificación definida, por lo que la propiedad evaluada se considera satisfecha. Los escenarios 5, 6 y 7 no buscan demostrar una violación, sino verificar que ante condiciones anómalas el controlador mantiene las propiedades de seguridad, vivacidad y temporales.

12. Propiedades verificadas

Propiedad	Escenario(s)
Propiedades de seguridad	
La bomba no debe administrar medicación si la última orden médica indica caudal igual a cero.	3
El caudal administrado no debe superar el máximo permitido.	1, 2, 3, 4, 5, 6, 7
Luego de una alarma crítica, la bomba no debe reanudar la infusión hasta recibir una confirmación del enfermero o una nueva orden médica.	5, 7
Propiedades de vivacidad	
Toda orden médica recibida debe producir eventualmente una acción sobre la bomba.	1, 2, 3
Toda alarma crítica no confirmada debe repetirse periódicamente.	7
Luego de detectar fin de bolsa, la infusión debe detenerse eventualmente.	6
Propiedades temporales	
Toda orden médica con caudal mayor que cero debe provocar el inicio de la infusión en menos de 3 segundos.	1, 2
Si el caudal difiere más del 10 % respecto del caudal indicado durante más de 5 segundos, debe emitirse una alarma media.	5, 7
Si se detecta fin de bolsa, la bomba debe detenerse como máximo luego de 60 segundos.	6
Si una alarma crítica no es confirmada dentro de 30 segundos, debe repetirse cada 10 segundos.	7

Cuadro 3: Verificación de las propiedades especificadas mediante los escenarios de simulación.

Del análisis de los resultados obtenidos en los distintos escenarios de simulación, se observa que todas las propiedades de seguridad, vivacidad y temporales definidas en la especificación fueron verificadas satisfactoriamente. Cada propiedad fue validada mediante uno o más escenarios específicos, permitiendo comprobar el comportamiento esperado del sistema tanto en condiciones normales como ante situaciones de falla.

13. Métricas obtenidas

A partir de las trazas generadas durante las simulaciones, se calcularon las principales métricas solicitadas para evaluar el comportamiento temporal del sistema.

Tiempo de respuesta ante desvíos de caudal

En los escenarios donde se simuló un desvío persistente de caudal, el caudal indicado fue de 100 ml/h y el caudal informado por el sensor fue de 80 ml/h. Esto representa un desvío del 20

$$\frac{|100 - 80|}{100} = 0,2 = 20$$

En la simulación, el desvío comenzó a ser informado por el sensor en $t = 1$ s y la alarma media fue emitida en $t = 6$ s. Por lo tanto, el tiempo de respuesta ante el desvío fue:

$$6 - 1 = 5 \text{ s}$$

Este resultado verifica que el sistema emite una `alarmaMedia` luego de que el desvío persiste durante más de 5 segundos.

Tiempo de respuesta ante fin de bolsa

El evento `finBolsa` fue detectado en $t = 37.5$ s. El controlador, cumpliendo exactamente el límite especificado, emitió la orden `detenerBomba` en $t = 97.5$ s (60.0 s después de la detección). El actuador aplicó dicho cambio 0.5 s más tarde, llevando el caudal real a 0 ml/h en $t = 98.0$ s, conforme a su latencia máxima especificada (Cuadro 1). El intervalo total hasta la detención física (60.5 s) se descompone entonces de manera exacta en los 60.0 s de decisión del controlador más los 0.5 s de latencia del actuador, sin margen de aproximación.

Cantidad de detenciones preventivas

La cantidad de detenciones preventivas fue obtenida a partir del archivo `registro_eventos.txt`. Los resultados para los escenarios simulados fueron los siguientes:

Escenario	Descripción	Detenciones
1	Funcionamiento normal sin fallas	0
2	Cambio de orden médica durante la infusión	0
3	Orden médica con caudal igual a cero	1
4	Desvío leve de caudal	0
5	Desvío mayor al 10 % durante más de 5 segundos	1
6	Fin de bolsa con confirmación del enfermero	1
7	Alarma crítica no confirmada durante 30 segundos	1

Cuadro 4: Cantidad de detenciones registradas en cada escenario de simulación.

Los resultados muestran que las detenciones se producen únicamente en escenarios donde corresponde detener la bomba: ante una orden médica de caudal cero, ante una alarma crítica o ante la detección de fin de bolsa.

Porcentaje de tiempo con infusión correcta

Se definió como infusión correcta a toda condición en la cual el caudal real administrado se mantiene dentro del rango permitido respecto del caudal indicado por la orden médica. De acuerdo con la especificación, el desvío máximo aceptable es del 10 %.

El porcentaje de tiempo con infusión correcta se calculó mediante la siguiente expresión:

$$\text{Porcentaje de infusión correcta} = \frac{\text{Tiempo con } |cReal - cObj| \leq 0,1 \cdot cObj}{\text{Tiempo total de simulación}} \times 100$$

La Tabla 5 resume los resultados obtenidos para los distintos escenarios simulados.

Escenario	Descripción	% de infusión correcta
1	Funcionamiento normal sin fallas	100
2	Cambio de orden médica durante la infusión	100
3	Orden médica con caudal igual a cero	100
4	Desvío leve de caudal	100
5	Desvío mayor al 10 % durante más de 5 segundos	18
6	Fin de bolsa con confirmación del enfermero	81
7	Alarma crítica no confirmada durante 30 segundos	9

Cuadro 5: Porcentaje de tiempo con infusión correcta para cada escenario.

Estos resultados permiten evaluar la capacidad del sistema para mantener el caudal administrado dentro de los límites establecidos por la especificación.

14. Propuesta de mejora

Una posible mejora del controlador consiste en incorporar un mecanismo de corrección automática del caudal ante desvíos detectados. En la implementación actual, cuando el caudal real difiere más de un 10 % respecto del caudal indicado, el controlador espera que el desvío persista durante un intervalo determinado, emite una alarma media y, si la situación continúa, escala a una alarma crítica y detiene la bomba.

Como mejora, el controlador podría calcular explícitamente el error entre el caudal objetivo y el caudal real informado por el sensor:

$$error = cObj - cReal$$

A partir de este error, podría generarse una nueva orden de ajuste de caudal proporcional al desvío observado:

$$nuevoCaudal = cObj + K_p \cdot error$$

donde K_p representa una ganancia proporcional configurable. De esta manera, el sistema no sólo detectaría la existencia de un desvío, sino que intentaría corregirlo automáticamente antes de llegar a una condición crítica.

Esta mejora permitiría reducir la cantidad de detenciones preventivas, aumentar el porcentaje de tiempo con infusión correcta y disminuir el riesgo de que una desviación persistente derive directamente en una alarma crítica. Además, el criterio de corrección dejaría de depender únicamente de tiempos fijos y pasaría a considerar la magnitud real del error observado.

15. Conclusiones

A partir de la descripción del problema, se desarrolló una especificación formal utilizando el formalismo DEVS y posteriormente se implementó el sistema en PowerDEVS. Las simulaciones realizadas permitieron analizar el comportamiento de la bomba de infusión tanto en condiciones normales como ante diferentes situaciones de falla.

Los resultados obtenidos verificaron el cumplimiento de las propiedades de seguridad, vivacidad y los requisitos temporales establecidos en la consigna. Además, el análisis realizado permitió identificar posibles mejoras para incrementar la robustez del sistema.

En conclusión, el trabajo permitió modelar, implementar y validar un sistema de tiempo real mediante simulación de eventos discretos, obteniendo resultados consistentes con los objetivos planteados.